Compiler Construction

Chapter 1

Compilers: Introduction and Motivation

Clemens Grelck

Friedrich Schiller University Jena

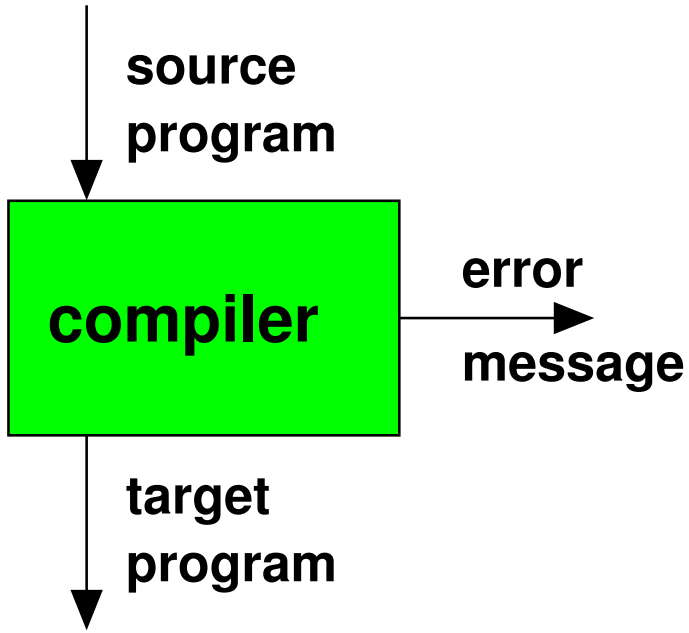
What is a Compiler ?

Definition:

- A compiler is an executable program
- that translates
- textual programs
- represented in a source language
- into semantically equivalent
- textual programs
- represented in a destination language
- (if possible)
- following some motivation.

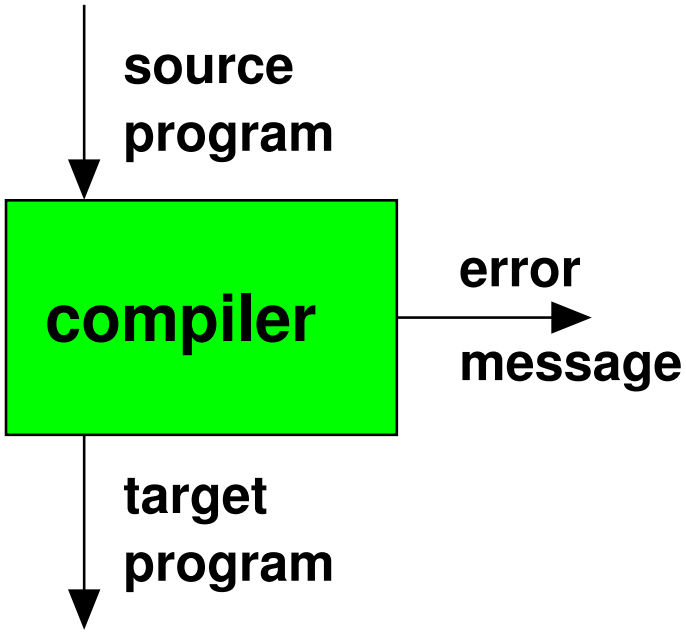
What is a Compiler ?

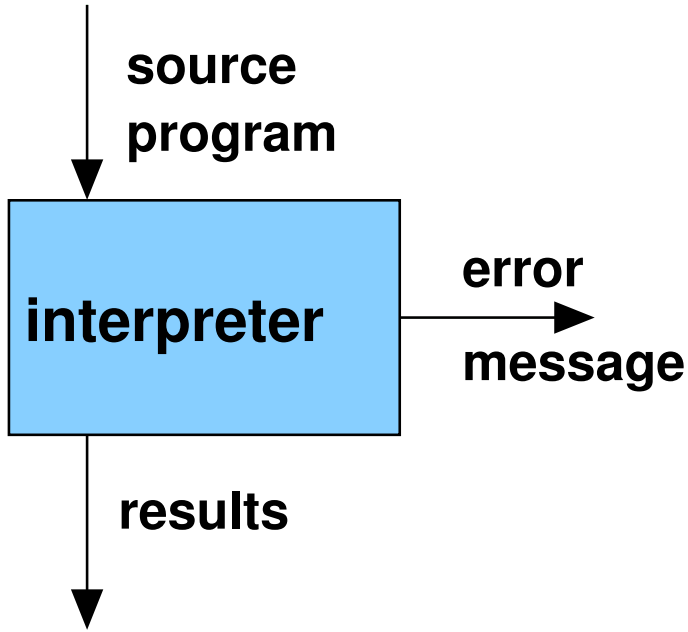
Illustration:



Compiler vs Interpreter

Comparison:





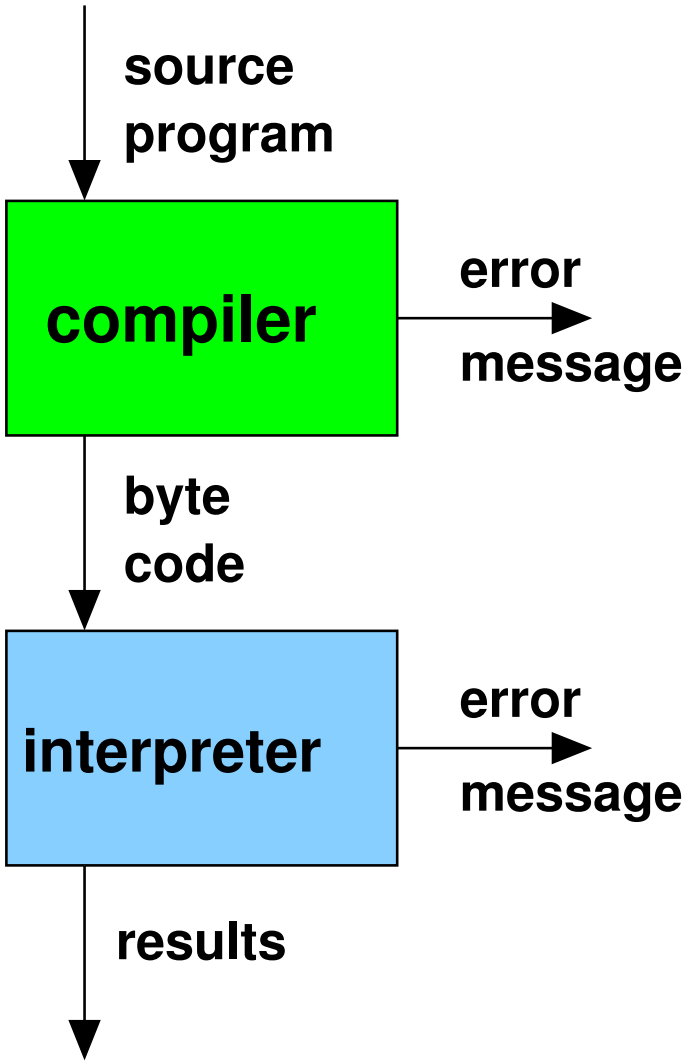
Similarities:

- Analysis of source code
- Optimisation

Differences:

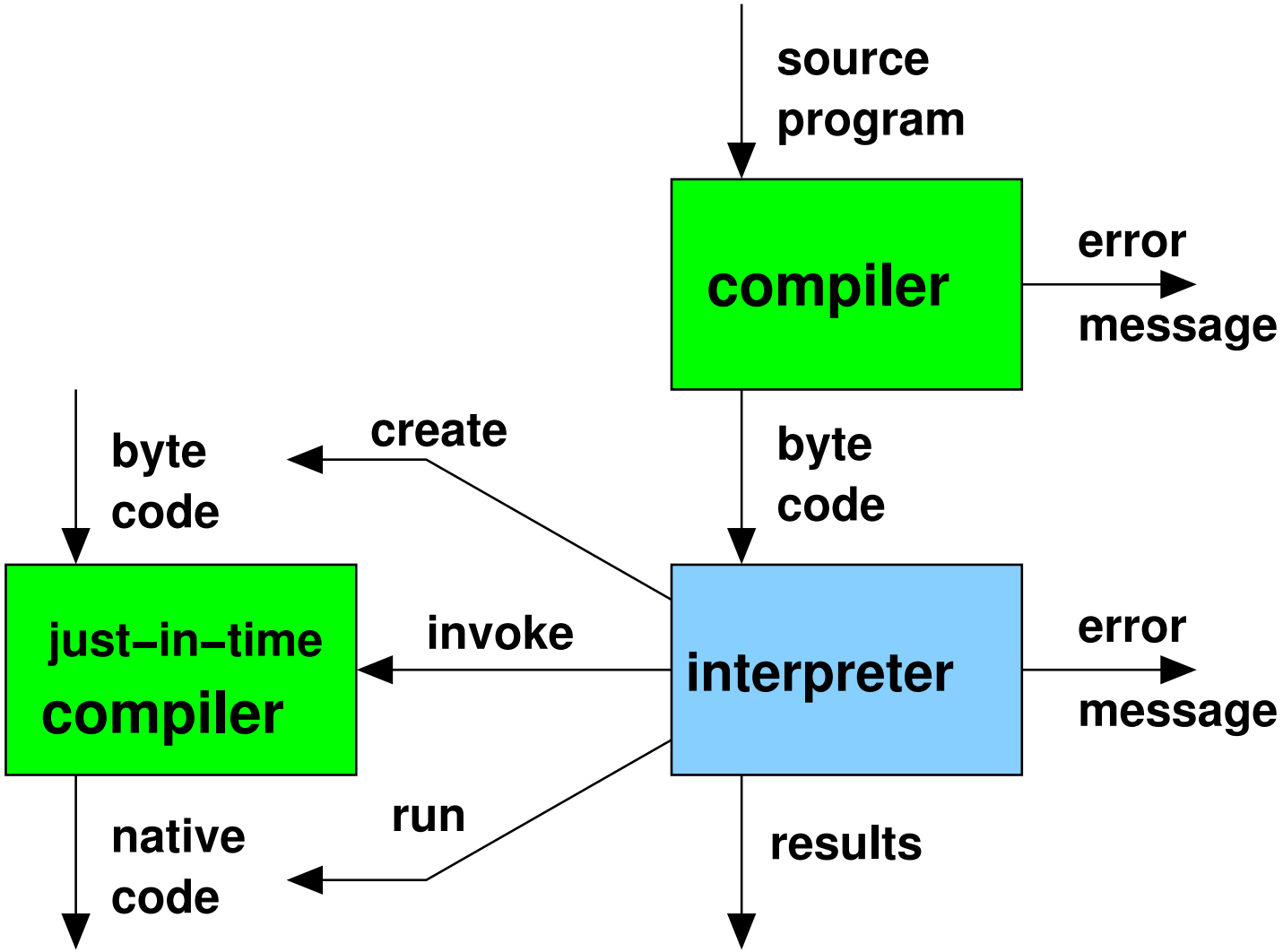
- Interpreter *executes* code.
- Compiler *describes* execution.

Hybrid Architecture: Compiler and Interpreter



- Compiler produces intermediate code for virtual machine
- Interpreter implements virtual machine on real hardware
- Popular approach since the advent of Java
- Advantage: code portability and code motion
- Disadvantage: less efficient than native code

Just-in-Time Compilation (JIT)

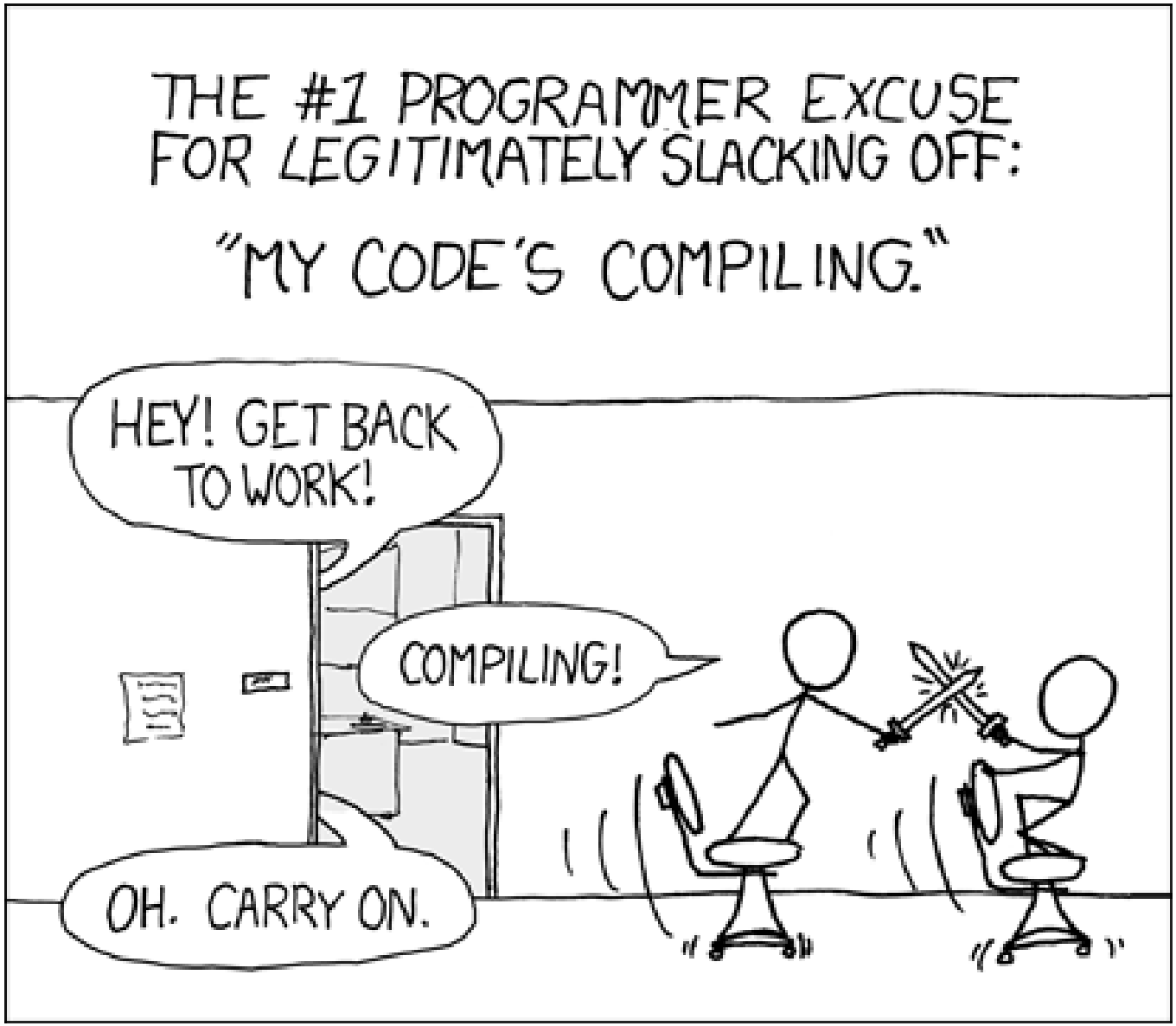


Just-in-Time Compilation (JIT)

Characteristics:

- Interpreter invokes (just-in-time) compiler on byte code sections
- Instead of interpreting the byte code, the interpreter runs the native code
- Compilation overhead must amortise !
- Desirable: compile once, run zillion times

Compilation Times



Offline vs Online Algorithms

Reminder:

- Offline algorithm:
  complete input available at startup
- Online algorithm:
  input becomes available during execution

Where are compilers ?

- Compilers employ classic offline algorithms
- Explicit *phase distinction*:
  - static / compile time
  - dynamic / run time
- With just-in-time compilation the boundaries become fuzzy
- With syntax completion even more ...

First Fundamental Principle of Compilation

Semantic equivalence !

Source and target program must be semantically equivalent:

- Obvious isn't it ?
- Otherwise, we could generate **any** target program, including `noop` and `print(42)`.

First Fundamental Principle of Compilation

Semantic equivalence !

Source and target program must be semantically equivalent:

- Obvious isn't it ?
- Otherwise, we could generate **any** target program, including `noop` and `print(42)`.

Less trivial to achieve in practice:

- Semantics of source language formally defined ?
- Semantics of target language formally defined ?
- What about the notorious odd cases ?

Second Fundamental Principle of Compilation

There must be a motivation !

Principle:

- Programs in source language must be available somehow.
- Programs in target language must be desirable somehow.

Second Fundamental Principle of Compilation

There must be a motivation !

Motivation: Abstraction lowering:

- Source language is more amenable to programming.
- Target language is more amenable to execution.
- Complex language features in the source language are replaced by composition of simpler ones in the target language.

Second Fundamental Principle of Compilation

There must be a motivation !

Motivation: Improvement on non-functional properties:

- Source and target language (merely) identical
- Optimisation for:
  speed, power, code size, memory footprint, ...
- Source-to-source translator

Second Fundamental Principle of Compilation

There must be a motivation !

Motivation: Feature reduction:

- Replace complex features in source language by simpler features again in source language
- "Desugaring"
- Reduction to core language

Compilers Are Everywhere (Actually)

Traditional compilers:

- Implement fully-fledged programming languages
- Target (virtual) machine code

Compilers Are Everywhere (Actually)

Traditional compilers:

- Implement fully-fledged programming languages
- Target (virtual) machine code

Research compilers:

- Support high-level (experimental?) programming features
- Often target other languages:
  - core host language
  - C as portable, high-level assembly language
  - LLVM (Low Level Virtual Machine)
  - JVM (Java Virtual Machine)

Compilers Are Everywhere (Actually)

Not the usual suspects:

- Type setting systems: tex, latex
- Bibliography systems: bibtex
- graphics converter: imagemagic, gif2png, fig2dev, ...
- page layout description converter: ps2pdf, pdf2ps, ...
- ...

Compilers Are Everywhere (Actually)

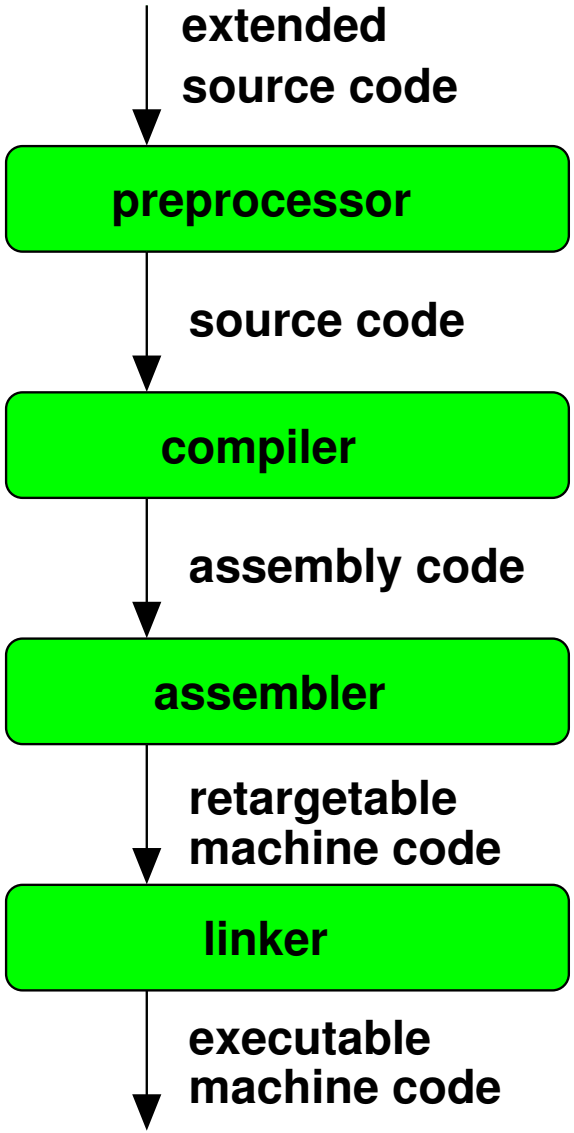
Not the usual suspects:

- Type setting systems: tex, latex
- Bibliography systems: bibtex
- graphics converter: imagemagic, gif2png, fig2dev, ...
- page layout description converter: ps2pdf, pdf2ps, ...
- ...

The truth is:

- Any processing of structured text requires a compiler !!

Compiler Tool Chain



The truth is:

- Every stage is a *compiler*.
- The terms *source language* and *target language* are not absolute.
- The *target language* of one compiler may be the *source language* of another compiler.

The End: Questions ?

